



2services-structs

İşletim Sistemi Servisleri

Kullanıcıya ve Programlara Yardımcı Olan Servisler

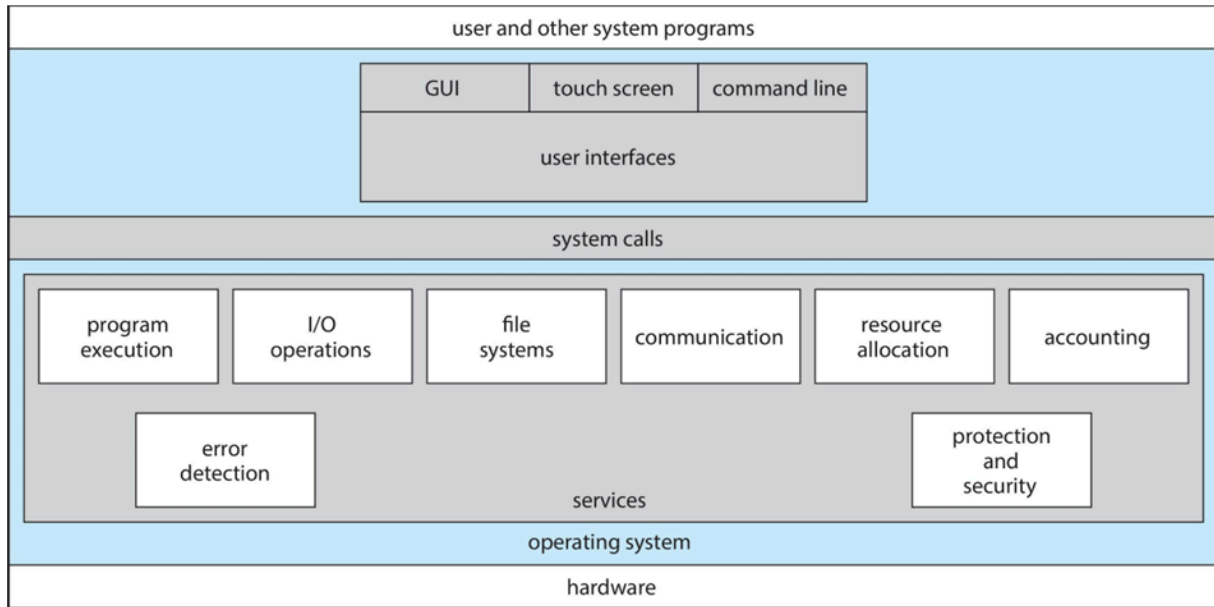
- **Kullanıcı Arayüzü (User Interface - UI):** Neredeyse tüm işletim sistemlerinde bulunur. Komut satırı (CLI), Grafik arayüz (GUI) veya Dokunmatik (Touch) olabilir.
- **Program Çalıştırma (Program Execution):** İşletim sistemi, yazdığın veya tıkladığın bir programı alıp belleğe (RAM) yüklemek, çalıştırmak ve işi bittiğinde (veya hata verdiğinde) sonlandırmak zorundadır.
- **I/O İşlemleri:** Çalışan bir programın diske bir şey yazması veya bir cihazdan (kamera, klavye) veri alması gerektiğinde bu işlemi OS yönetir.
- **Dosya Sistemi İşlemleri (File-system Manipulation):** Programların dosya ve klasörleri okuması, yazması, yaratması, silmesi ve bu dosyaların izinlerini (permissions) yönetmesi sistemin en önemli görevlerindendir.
- **İletişim (Communications):** Süreçlerin (process) kendi aralarında veya ağ üzerinden başka bilgisayarlardaki süreçlerle haberleşmesidir. İki yöntemle yapılır: Ya "Paylaşımlı Bellek (Shared Memory)" kullanılır ya da OS üzerinden "Mesajlaşma (Message Passing)" ile paketler taşınır.
- **Hata Tespiti (Error Detection):** İşlemcide, bellekte veya çalıştırılan bir kullanıcı programında anlık donanımsal/yazılımsal hatalar çıkabilir. OS bu hataları sürekli izler ve sistemin çökmemesi için uygun eylemi alır.

Sistemin Verimliliğini Sağlayan Servisler

- **Kaynak Tahsisi (Resource Allocation):** Sistemde aynı anda birden fazla kullanıcı veya program çalışırken; işlemci (CPU) döngülerini, RAM'i ve

donanımları kimin ne kadar kullanacağını OS dağıtır.

- **Kayıt Tutma (Logging):** Hangi kullanıcının veya programın, hangi kaynakları ne kadar tükettiğinin kaydını tutar.
- **Koruma ve Güvenlik (Protection and Security):**
 - *Koruma:* Sistemdeki süreçlerin birbirine karışmasını ve birbirini bozmasını engellemektir.
 - *Güvenlik:* Sistemi dışarıdan gelen (ağ üzerinden vb.) yetkisiz erişimlere ve saldırılara karşı savunmaktır.



Command Line Interpreter

Kullanıcının doğrudan sistem komutları girmesini sağlayan arayüzdür. Bazen OS çekirdeğinin (kernel) içine gömülüdür, bazen de ayrı bir sistem programı (Shell) olarak çalışır.

Örneğin terminale `rm file.txt` yazdığında, CLI (Shell) arka planda gider, `rm` isimli programı bulur, belleğe yükler ve parametre olarak verdiği "file.txt" dosyasını silmesi için o programı çalıştırır.

API

Programlar genelde doğrudan syscall kullanmak yerine API'lar üzerinden işlem yaparlar.

- API kullanmak kodun taşınabilir (portable) olmasını sağlar ve karmaşık donanım/çekirdek geçiş detaylarını programcıdan gizler.
- Sen C dilinde ekrana bir şey yazdırmak için `printf()` fonksiyonunu çağırdığında, bu standart C kütüphanesi (libc) içindeki ayrıcalıksız bir koddur . Bu kütüphane senin adına arka planda işletim sisteminin `write()` sistem çağrısını tetikler ve sonucu sana döndürür.



- Systemcall işlemleri basit bir işlem için bile onlarca defa dosya açıp kapama işlemi yaparlar bu yüzden doğru ve optimize yazılmış olmaları gerekir

Hardware Interrupt vs Software Interrupt (Trap) vs Exception

- **Hardware Interrupt (Donanım Kesmesi):** Dışarıdan, yani donanımdan gelen ve işlemcinin anında ilgilenmesi gereken asenkron/beklenmedik sinyallerdir.
 - Örneğin; farenin hareket etmesi veya klavyeden bir tuşa basılması .
- **Software Interrupt / Trap (Yazılım Kesmesi):** Bir uygulamanın **bilerek ve isteyerek** işletim sisteminden hizmet talep etmesidir.
 - Yaptığımız sistem çağrıları (syscall) aslında birer Trap'tir .
- **Exception (İstisna/Hata):** Kodun çalışması sırasında ortaya çıkan senkronize ve beklenmedik bir hatadır.
 - Örneğin; bir sayıyı sıfıra bölmeye çalışmak (division by zero) veya programın yetkisi olmayan bir bellek adresine erişmeye kalkması (page fault) .

İşletim Sistemine Parametre Gönderme Yöntemleri

1. Register (Yazmaç) Yöntemi:

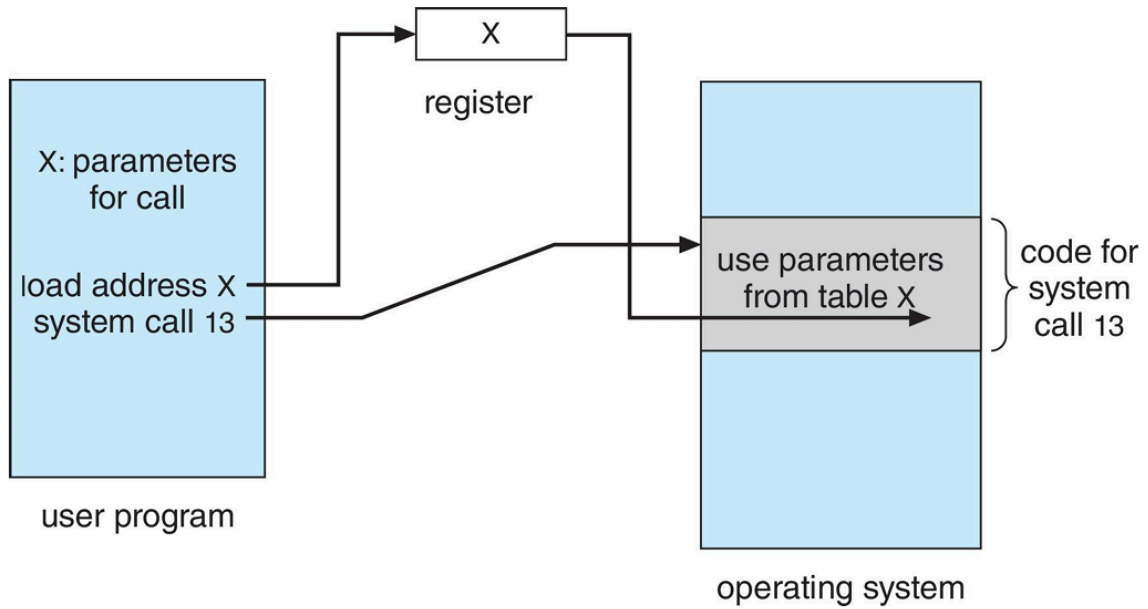
- a. Parametreler doğrudan CPU'nun içindeki süper hızlı yazmaçlara konur.
- b. En basit yoldur ama yazmaç sayısı az olduğu için parametre sayısı fazlaysa yetersiz kalır.

2. Block / Table (Blok/Tablo) Yöntemi:

- a. Parametreler bellekte (RAM) bir bloğa veya tabloya yazılır. OS'e sadece bu bloğun bellekteki "başlangıç adresi" tek bir yazmaç üzerinden iletilir .
- b. Parametre sayısı veya uzunluğu için hiçbir sınır yoktur.
- c. Linux ve Solaris bu yöntemi çok sık kullanır .

3. Stack (Yığın) Yöntemi:

- a. Program, göndermek istediği parametreleri yığına (stack) ekler (push) .
- b. İşletim sistemi de sırası geldiğinde bu parametreleri yığından çıkarıp (pop) kullanır.
- c. Bu da sınırları ortadan kaldıran oldukça esnek bir yöntemdir.

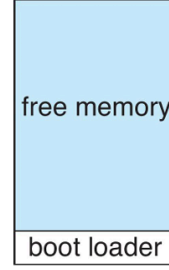


Uygulamalar Neden İşletim Sistemlerine Özeldir?

- **Farklı Sistem Çağrıları (API):** Her işletim sisteminin çekirdeği (kernel) farklıdır ve dolayısıyla sistem çağrıları da tamamen benzersizdir.
- **Farklı Dosya Formatları:** İşletim sistemleri, çalıştırılabilir dosyaları belleğe yüklemek için kendine has ikili (binary) dosya formatları kullanır.

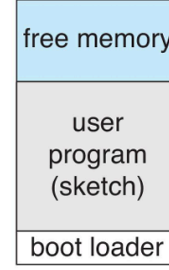
Süreç Çalıştırma (Single-tasking vs. Multi-tasking)

- **Tek Görevli (Single-tasking - Arduino Örneği):** Ortada bir işletim sistemi bile yoktur. Sadece ufak bir "Bootloader" (Önyükleyici) kodu vardır; yazdığın kodu alır ve tüm belleği ona tahsis ederek tek başına çalıştırır .



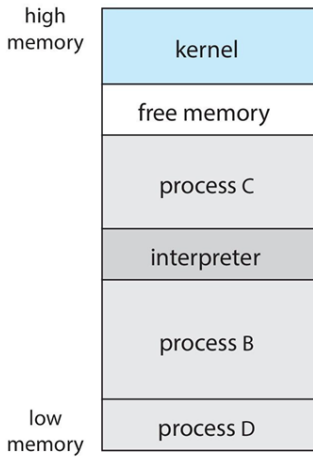
(a)

At system startup



(b)

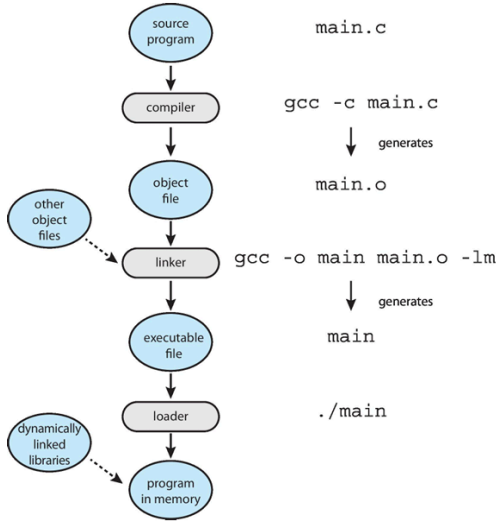
running a program



- **Çok Görevli (Multi-tasking - FreeBSD/UNIX Örneği):** Karmaşık bir süreç işler. Kullanıcı sisteme girdiğinde bir komut satırı (shell) açılır . Yeni bir program açılacağı zaman OS önce `fork()` komutuyla bir kopya süreç yaratır, sonra `exec()` komutuyla asıl programı bu sürecin içine yükler. Program bittiğinde ise `exit()` ile hata kodu döndürerek kapanır.

Linkers & Loaders

1. **Derleyici (Compiler):** Yazdığın kodu, işlemcinin anlayabileceği "Nesne Dosyalarına" (Object files - `.o` uzantılı) dönüştürür. Ancak bu dosya henüz tek başına çalışamaz.
2. **Bağlayıcı (Linker):** Nesne dosyalarını ve kodunda kullandığın kütüphaneleri (örneğin C'deki standart kütüphaneleri) birbirine bağlayarak tek bir **Çalıştırılabilir Dosya (Executable File)** oluşturur.
3. **Yükleyici (Loader):** İşletim sisteminin bir parçası olan bu araç, o çalıştırılabilir (.exe vb.) dosyayı diskten alır, **belleğe (RAM'e)** taşır ve çalışmaya hazır hale getirir.



Uygulamaları Çoklu İşletim Sistemine (Multi-OS) Uyumlu Hale Getirmek

- **Yorumlanan Diller Kullanmak:** Python veya Ruby gibi diller, makine koduna dönüşmeden anlık olarak okunur. Eğer o işletim sisteminde dili okuyacak bir "Yorumlayıcı (Interpreter)" kuruluysa, kod her yerde çalışır.
- **Sanal Makine (Virtual Machine) Kullanmak:** Java bunun en meşhur örneğidir. Kod, "Java Sanal Makinesi (JVM)" için derlenir. İçinde JVM olan her işletim sistemi o kodu çalıştırabilir.
- **ABI (Application Binary Interface) Seviyesinde Uyum:** İki farklı yazılımın (örneğin işletim sistemi ve programın) donanım seviyesinde nasıl konuşacağını tanımlayan standart arayüzdür.

İşletim Sistemi Tasarımı ve Uygulaması

Politika vs Mekanizma

- **Policy (Politika):** *Ne yapılacaktır?* Sorusunun cevabıdır.
 - Örneğin: Zamanlayıcı her 100 milisaniyede bir işlemciyi durdursun.
- **Mechanism (Mekanizma):** *Nasıl yapılacaktır?* Sorusunun cevabıdır.

- Örneğin: Bunu yapmak için donanımsal bir Timer (Zamanlayıcı) devresi kullanalım.

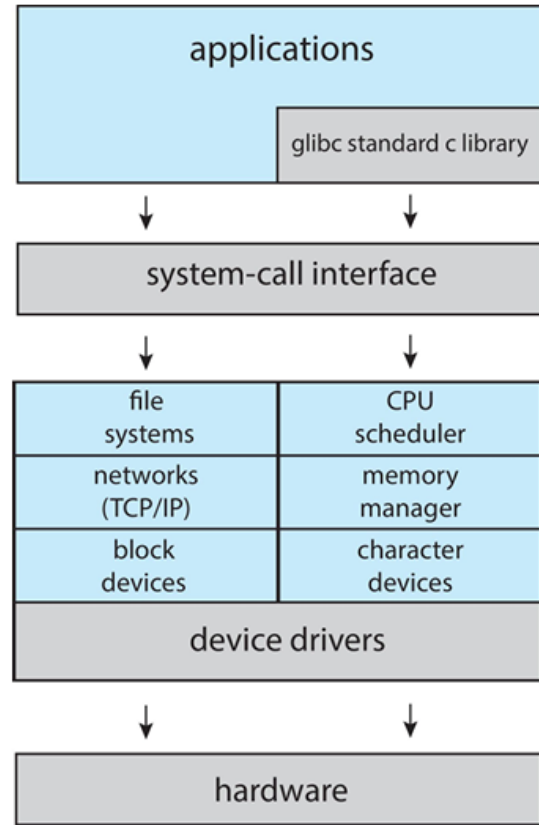
Neden ayırırız?

Maksimum esneklik sağlamak için! Eğer ileride 100 ms yerine 200 ms kullanmak istersen, sadece politikayı değiştirirsin, arka plandaki mekanizmayı (zamanlayıcı kodlarını) baştan yazmak zorunda kalmazsın.

İşletim Sistemi Mimarileri

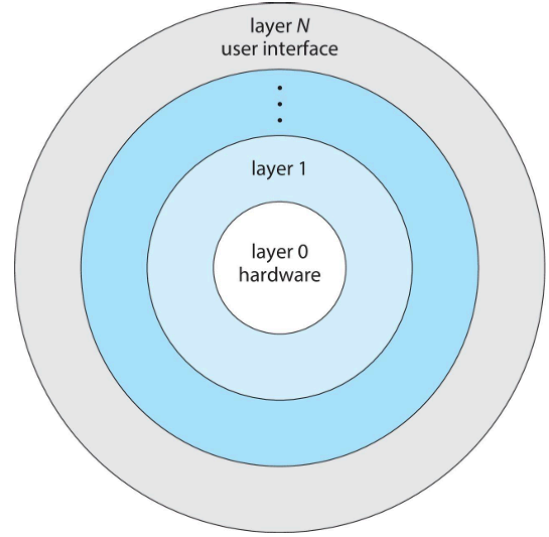
Monolitik (Tek Parça) Yapı

- Orijinal UNIX ve modern Linux bu yapıdadır.
- İşletim sisteminin tüm parçaları (dosya sistemi, CPU zamanlayıcısı, bellek yönetimi, cihaz sürücüler) tek bir devasa Çekirdeğin (Kernel) içine sıkıştırılmıştır.
- İnanılmaz **hızlıdır**. Çünkü her şey aynı kutunun içindedir ve birbirleriyle haberleşirken bir sınır geçmek zorunda kalmazlar (overhead yoktur).
- Karmaşıktır ve güvenli değildir. Ses kartı sürücüsünde (driver) ufak bir hata (bug) olursa, tüm işletim sistemi aynı kernel içinde olduğu için sistem tamamen çökebilir.



Katmanlı (Layered) Yapı

- Sistemi daha modüler hale getirmek için yapıyı soğan gibi katmanlara ayırma fikridir.
- En alt katman (Layer 0) doğrudan donanımdır, en üst katman (Layer N) ise kullanıcı arayüzüdür.
- Altın kural şudur: Her katman **sadece ve sadece** bir altındaki katmanın fonksiyonlarını kullanabilir.

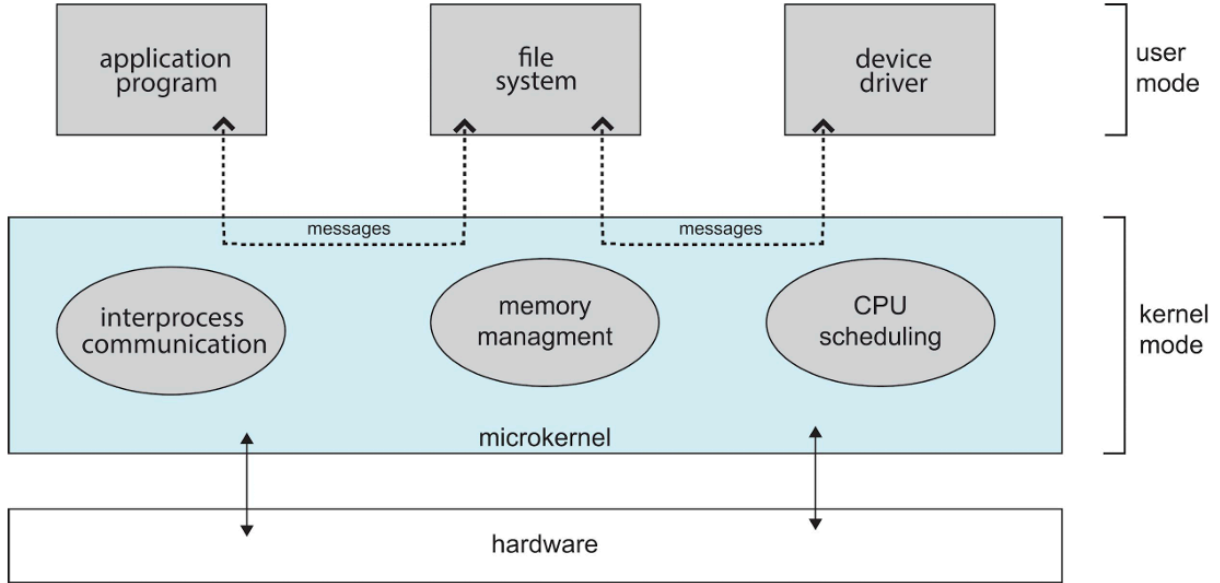


- Hata ayıklaması (debugging) çok basittir.
 - Örneğin; 1. katman çalışıyorsa ve 2. katmanda hata çıkarsa, hatanın tam olarak nerede olduğunu bilirsin.
- Performans düşüktür (bir verinin 5 katman aşağı inmesi zaman alır) ve katmanların sırasını belirlemek çok zordur.
 - Örneğin; bellek yönetimi mi daha altta olmalı, disk sürücüsü mü?

Mikroçekirdek (Microkernel) Mimarisi

- Monolitik yapının tehlikelerine karşı geliştirilmiştir.
- İşletim sisteminin hayati olmayan tüm bileşenleri (dosya sistemleri, cihaz sürücüler vb.) çekirdekten (kernel) çıkartılır ve sıradan bir programmış gibi Kullanıcı Moduna (User Mode) taşınır.
- Çekirdek çok küçülür (Micro). Tek yaptığı şey bellek/CPU yönetimi ve bu dışarı atılan servislerin birbiriyle mesajlaşmasını sağlamaktır.
- İnanılmaz **güvenilir ve güvenlidir** .
 - Ses kartı sürücüsü çökerse sadece o süreç kapanır, sistemin kalbi (kernel) çalışmaya devam ettiği için bilgisayar çökmez.
- **Dezavantajı:** Performans kaybı (Overhead).

- Cihaz sürücüleri artık dışarıda olduğu için, her işlemde sürekli User Mode ile Kernel Mode arasında mesajlaşma gidip gelmek zorundadır, bu da sistemi yavaşlatır.



Sistemin Başlatılması (System Boot)

- Güç verildiğinde sistem, işlemcinin içine donanımsal olarak kazınmış sabit bir bellek konumundan (ROM veya EEPROM) çalışmaya başlar.
- **Bootstrap Loader (Önyükleyici):** Bu donanımsal bellekte bulunan küçük kod parçasına (eski adıyla BIOS, modern adıyla UEFI) önyükleyici denir.
 - Bunun tek bir görevi vardır: İşletim sisteminin çekirdeğini (kernel) diskte bulmak, RAM'e yüklemek ve onu başlatmak.
 - Bazen bu işlem iki adımda yapılır. ROM'daki kod sadece diskin en başındaki (boot block) ufak bir kodu çağırır, o ufak kod da asıl büyük önyükleyiciyi yükler.
- **GRUB:** Linux dünyasında çok meşhur olan GRUB gibi gelişmiş önyükleyiciler, bilgisayar açılırken sana hangi işletim sistemini veya hangi çekirdek (kernel) versiyonunu açmak istediğini soran o siyah ekranı sunar.

Çekirdek (Kernel) RAM'e yüklenip çalışmaya başladığında artık sistem ayağa kalkmış demektir.

Hata Ayıklama (OS Debugging)

- **Core Dump (Çekirdek Dökümü):** Eğer senin yazdığın normal bir uygulama (program) çökerse, işletim sistemi o programın o anki bellek (RAM) durumunu kopyalar ve bir "core dump" dosyasına yazar. Bu sayede programcı, kodun neresinde patladığını görebilir.
- **Crash Dump (Çökme Dökümü):** Eğer çöken şey sıradan bir program değil de **İşletim Sisteminin kendisiyse** (Kernel çöktüyse), sistem tüm çekirdek belleğini bir "crash dump" dosyasına kaydeder

Performans İyileştirme ve İzleme

- **strace** : Bir programın arka planda işletim sistemine hangi **sistem çağrılarını (system calls)** yaptığını adım adım izler.
- **gdb** : Kaynak kod seviyesinde adım adım hata ayıklayıcıdır.
- **tcpdump** : Ağ (Network) üzerinden geçen veri paketlerini yakalar ve analiz eder.

Gelişmiş Teşhis Aracı BCC

- Kullanıcı programları ile Çekirdeğin (Kernel) birbiriyle nasıl konuştuğunu analiz etmek çok zordur.
- Linux dünyasında bunun için **BCC** adında inanılmaz detaylı bir araç kiti kullanılır.